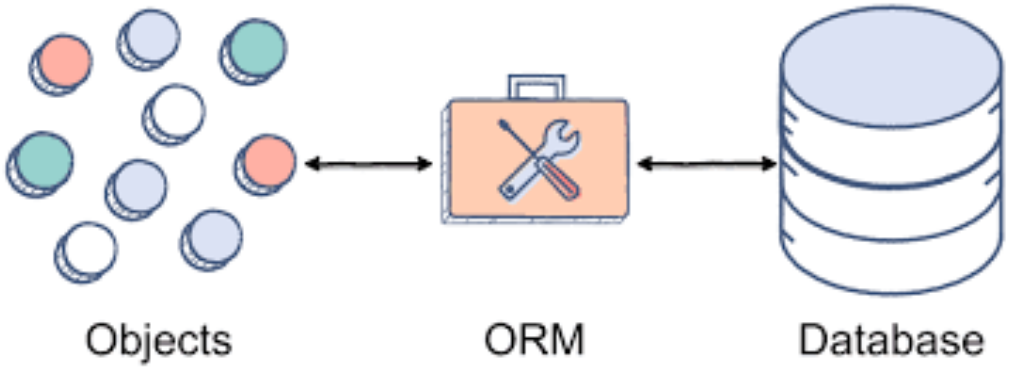
Object-Relational Mapper (ORM)

Part 2



22.04.26

Table of Contents

Database Exceptions

Why Handle Database Exceptions?

- Database operations can fail for many reasons:
  - Database file not found or inaccessible
  - Constraint violations (e.g. duplicate primary key)
  - Invalid SQL syntax
- Unhandled exceptions crash the program
- The **sqlite3** module defines a base **sqlite3.Error** exception for all database errors
- Use **try / except / finally** to handle errors gracefully

sqlite3 Exception Handling Pattern

```
conn = None
try:
    conn = sqlite3.connect(DatabaseName)
    cur = conn.cursor()
    # Perform database operations here.

except sqlite3.Error as err:
    print(err)

except Exception:
    # Handle general (non-DB) exceptions.

finally:
    if conn != None:
        conn.close()
```

Code Example: Exception Handling

```
def add_item(item_id, name, price):
    conn = None
    try:
        conn = sqlite3.connect('inventory.db')
        cur = conn.cursor()
        cur.execute('''INSERT INTO Inventory
            (ItemID, ItemName, Price)
            VALUES (?, ?, ?)''',
            (item_id, name, price))
        conn.commit()
    except sqlite3.Error as err:
        print(err)
    finally:
        if conn != None:
            conn.close()
```

SQLModel Exception Handling

sqlite3

```
conn = None
try:
  conn = sqlite3.connect('db')
  cur = conn.cursor()
  cur.execute(sql, params)
  conn.commit()
except sqlite3.Error as e:
  print(e)
finally:
  if conn != None:
    conn.close()
```

SQLModel

```
try:
  with Session(engine) as s:
    s.add(item)
    s.commit()
except Exception as e:
  print(e)

# with auto-closes Session
# __exit__ calls rollback()
# on unhandled exceptions
```

`Session` as context manager: auto-closes on exit, auto-rolls back on unhandled exceptions. No **`finally`** block needed!

SQLModel Exception Handling Patterns

A) Explicit rollback

```python
with Session(engine) as s:
  try:
    s.add(item)
    s.commit()
  except Exception as e:
    s.rollback()
    print(e)
```

B) Auto-rollback

```python
try:
  with Session(engine) as s:
    s.add(item)
    s.commit()
except Exception as e:
  print(e)

# __exit__ within Session auto-
rollback. No manual rollback
```

C) Minimal (propagate)

```python
with Session(engine) as s:
  s.add(item)
  s.commit()

# If commit() raises,
# __exit__ within Session rolls
back and re-raises the exception
and bubbles it up to whatever
called this code.

# If nothing catches the exception
the program will crash.
```

- Pattern **A**: Full control: explicit rollback inside the **`with`** block
- Pattern **B**: **`try`** wraps **`with`** - **`Session.__exit__()`** auto-rolls back before **`except`** runs
- Pattern **C**: Simplest:. Let the exception propagate; **`with`** handles cleanup automatically

Relational Data

Why Relations? The Data Duplication Problem

Storing all data in one table leads to problems:

- Duplicate data wastes storage space
- Inconsistent and conflicting information
- Updating one value requires changing many rows

EmployeeID	EmployeeName	DepartmentName	LocationName
1	Alice	Sales	Zurich
2	Bob	Sales	Zurich
3	Carol	HR	Geneva
4	David	HR	Geneva
5	Eve	IT	Basel
6	Mark	IT	Basel
7	Natalie	Marketing	Basel
8	Sandra	Marketing	Basel

What do you have to change when Alice switches to HR?

What do you have to change when Marketing is now in Zurich?

How many rows do you have to change when we change “Marketing” to “Marketing & Communications”

Solution: Normalization

Separate data into multiple related tables. Each table stores one type of entity. Example: Instead of one Employee table with repeated department names, split into **Employees**, **Departments**, and **Locations**

Before split (normalization)

EmployeeID	EmployeeName	DepartmentName	LocationName
1	Alice	Sales	Zurich
2	Bob	Sales	Zurich
3	Carol	HR	Geneva
4	David	HR	Geneva
5	Eve	IT	Basel

After split (normalization)

EmployeeID	EmployeeName	DepartmentID	LocationID
1	Alice	10	100
2	Bob	10	100
3	Carol	20	200
4	David	20	200
5	Eve	30	300

DepartmentID	DepartmentName
10	Sales
20	HR
30	IT

LocationID	LocationName
100	Zurich
200	Geneva
300	Basel

Foreign Keys

A **foreign key** is a column in one table that references a primary key in another table Creates a **relationship** between two tables

SQL syntax to create a foreign key:

```
CREATE TABLE Employees(
  EmployeeID INTEGER PRIMARY KEY,
  EmployeeName TEXT, DepartmentID INTEGER,
  LocationID INTEGER,
  FOREIGN KEY(DepartmentID)
    REFERENCES Departments(DepartmentID),
  FOREIGN KEY(LocationID)
    REFERENCES Locations(LocationID))
```

- Ensures **referential integrity** between tables

FK FK

EmployeeID	EmployeeName	DepartmentID	LocationID
1	Alice	10	100
2	Bob	10	100
3	Carol	20	200
4	David	20	200
5	Eve	30	300

Employees

PK

DepartmentID	DepartmentName
10	Sales
20	HR
30	IT

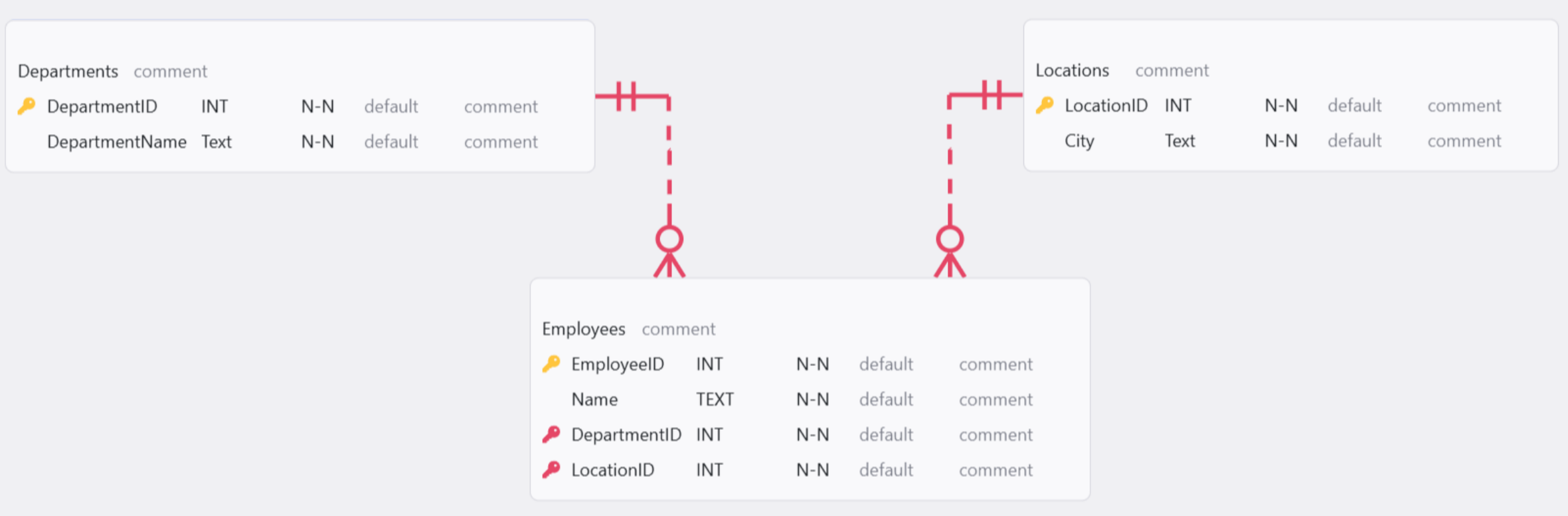
Departments

PK

LocationID	LocationName
100	Zurich
200	Geneva
300	Basel

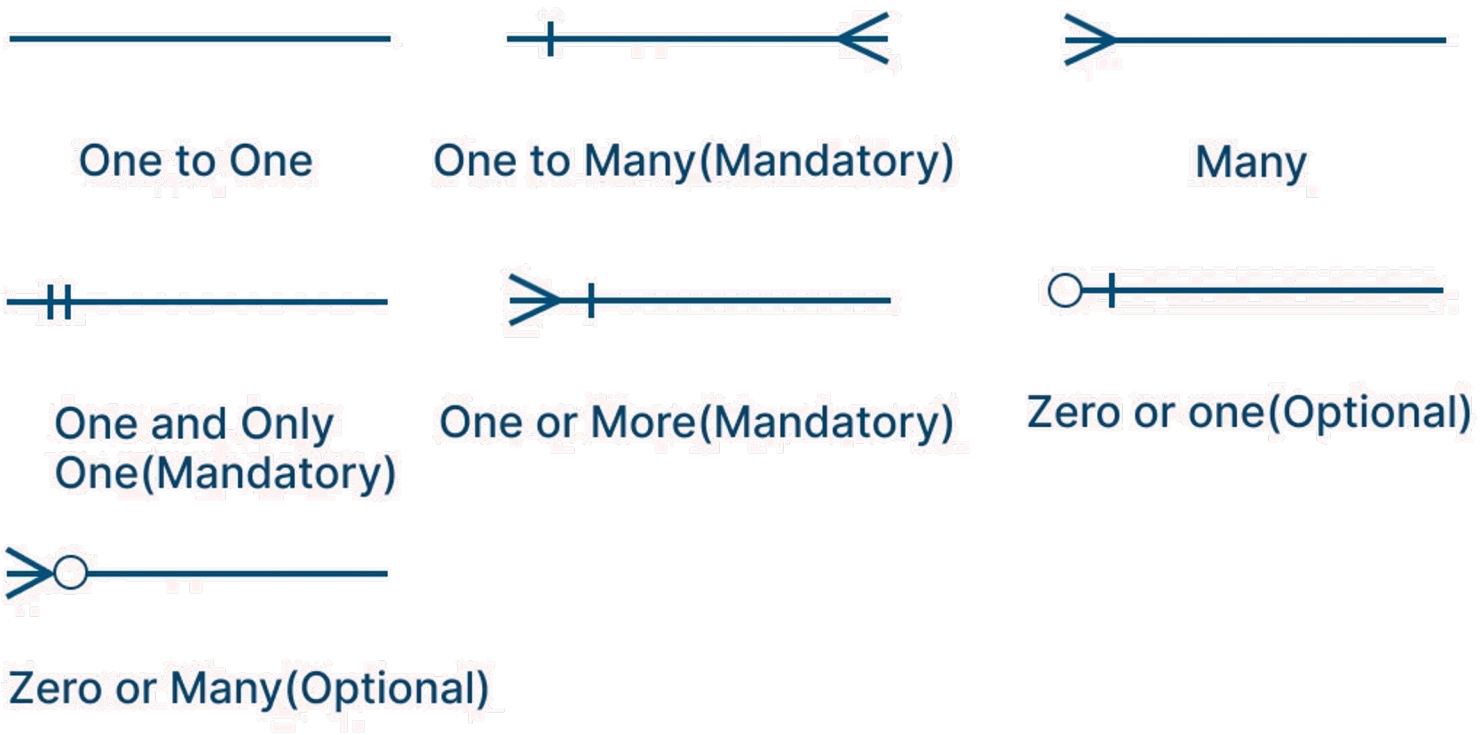
Locations

Entity Relationship Diagrams



You can install extensions for VS Code to create your ER Diagrams (search for ERD) or use online tools (e.g., erd-editor-app).

Entity Relationship Diagram-Symbols



Source: ER Diagram Symbols Explained | A Beginner's Guide | Creately

Relations in SQLModel

```python
from sqlmodel import SQLModel, Field, Relationship

class Department(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    employees: list["Employee"] = Relationship(back_populates="department")

class Employee(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    department_id: int = Field(foreign_key="department.id")
    department: Department | None = Relationship(back_populates="employees")
```

Multi-Table Queries

SQL (Qualified Column Names)

```
SELECT
  Employees.Name,
  Departments.DepartmentName,
  Locations.City
FROM
  Employees, Departments, Locations
WHERE
  Employees.DepartmentID == Departments.DepartmentID
AND
  Employees.LocationID == Locations.LocationID
```

SQLModel (Relationships)

```
with Session(engine) as s:
  employees = s.exec(
    select(Employee)
  ).all()

  for emp in employees:
    print(emp.name)
    print(emp.department.name)
    print(emp.location.city)

# No JOIN needed!
# SQLModel resolves relationships automatically
```

Foreign Keys in SQLite

SQLite does **not** enforce foreign keys by default!

Enable enforcement explicitly:

```
cur.execute('PRAGMA foreign_keys=ON')
```

What happens when a FK constraint is violated

```
>>> cur.execute('''INSERT INTO Employees
...    (Name, Position, DepartmentID, LocationID)
...    VALUES ("Bill", "Intern", 99, 100)''')

sqlite3.IntegrityError:
  FOREIGN KEY constraint failed
```

Also applies to UPDATE and DELETE operations

`FOREIGN KEY constraint failed` - Reasons

INSERT: FK values must match an existing PK in the referenced table

DepartmentID=99 fails if no department with ID 99 exists

UPDATE: Cannot change a FK to an invalid value

SET DepartmentID = 99 throws IntegrityError

DELETE: Cannot delete a row that is still referenced by another table

DELETE FROM Locations WHERE LocationID == 100 fails if employees reference it

SQLModel: Same rules apply: the database engine enforces FK constraints

Best Practices: Project Structure

Project Folder Structure

```
my_project/
├── database.py
├── models.py
├── dao/
│   ├── __init__.py
│   ├── department_dao.py
│   └── employee_dao.py
├── main.py
└── requirements.txt
```

`database.py` Engine setup, session factory, connection config

`models.py` Model classes with **SQLModel or without when using *DAOs*.** Note: model classes can also be stored in separated files (e.g. department.py, employee.py etc. instead of models.py).

`dao/` DAO (Data Access Object) classes: one per model, CRUD methods

`main.py` Entry point, user interface, calls CRUD

`requirements.txt` Dependencies (sqlmodel, etc.)

Key principle: Separation of concerns, each file has one clear responsibility

database.py - Engine & Session Setup

```python
from sqlmodel import SQLModel, Session, create_engine

DATABASE_URL = "sqlite:///my_app.db"

engine = create_engine(DATABASE_URL, echo=False)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

models.py - SQLModel Class Definitions

```python
from sqlmodel import SQLModel, Field, Relationship

class Department(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    employees: list["Employee"] = Relationship(
        back_populates="department")

class Employee(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    position: str
    dept_id: int = Field(foreign_key="department.id")
    department: Department | None = Relationship(
        back_populates="employees")
```

dao/ - **D**ata **A**ccess **O**bject Pattern

A DAO is a class or component that acts as the bridge between your program and the database, hiding the database details and giving the app simple methods like get, save, update, and delete. Like a waiter talking to the guest and the kitchen in a restaurant.

Application
uses data

DAO
fetches/saves data

DB
stores data

DAO separates database code from business logic

department_dao.py - Data Access Object Pattern

```python
from sqlmodel import Session, select
from models import Department

class DepartmentDAO:
    def __init__(self, session: Session):
        self.session = session

    def create(self, name: str) -> Department:
        dept = Department(name=name)
        self.session.add(dept)
        self.session.commit()
        self.session.refresh(dept)
        return dept

    def get_by_id(self, id: int):
        return self.session.get(Department, id)

    def get_all(self) -> list[Department]:
      return self.session.exec(select(Department)).all()

    def delete(self, dept: Department):
       self.session.delete(dept)
       self.session.commit()
```

employee_dao.py - With Foreign Keys

```python
from sqlmodel import Session, select
from models import Employee, Department

class EmployeeDAO:
    def __init__(self, session: Session):
        self.session = session

    def create(self, name: str, position: str,
               dept_id: int) -> Employee:
        # dept_id must match existing Department.id
        emp = Employee(name=name, position=position,
                       dept_id=dept_id)
        self.session.add(emp)
        self.session.commit()
        self.session.refresh(emp)
        return emp
```

```python
    def get_by_department(self, dept_id: int):
        # Uses relationship — no manual JOIN
        dept = self.session.get(Department, dept_id)
        return dept.employees if dept else []
```

main.py - Putting It All Together

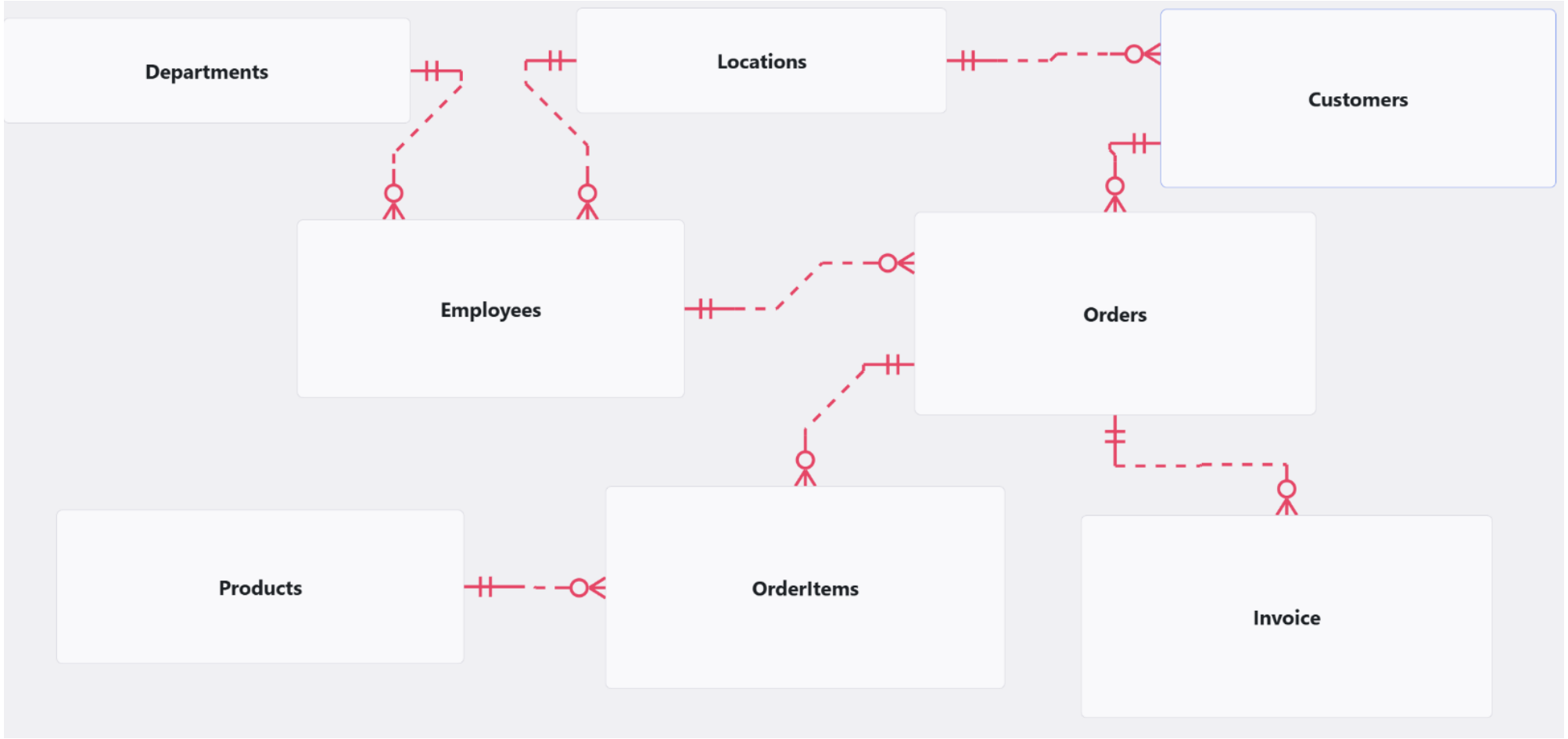
```python
from database import create_db_and_tables, engine
from dao import DepartmentDAO, EmployeeDAO
from sqlmodel import Session

def main():
    create_db_and_tables()
    with Session(engine) as session:
        dept_dao = DepartmentDAO(session)
        emp_dao = EmployeeDAO(session)

        dept = dept_dao.create("Engineering")
        emp = emp_dao.create("Alice", "Dev", dept.id)

        print(emp.department.name)
        # -> "Engineering"
```

Summary

- Always wrap database operations in **try / except / finally**
- Catch **sqlite3.Error** for database-specific exceptions
- SQLModel simplifies exception handling with **Session** context managers
- Normalize data into separate tables to avoid duplication
- **Foreign keys** create relationships between tables and ensure referential integrity
- SQLite requires **PRAGMA foreign_keys=ON** to enforce FK constraints
- SQLModel uses **Field(foreign_key=...)** and **Relationship()** for Pythonic access to related data
- Sample project structure to organize the code

Addon: Example of a larger ERD

Larger ER Diagram



Larger ER Diagram